24/07/2025

TASK-I

A temple ticket online booking Management system

A temple ticket online booking management system allows devotes to book tickets for temple visits special darshan, poojas, and other events onlines, reducing physical queoves and improving crow-d management. These system oftem include teatues like date and time slot seullection, Payment Processing and the generation of tickets or Passes.

Entity:-

The real-world object or concept that can be distinctly identified. Examples included students, courses, or products.

Entity set:-

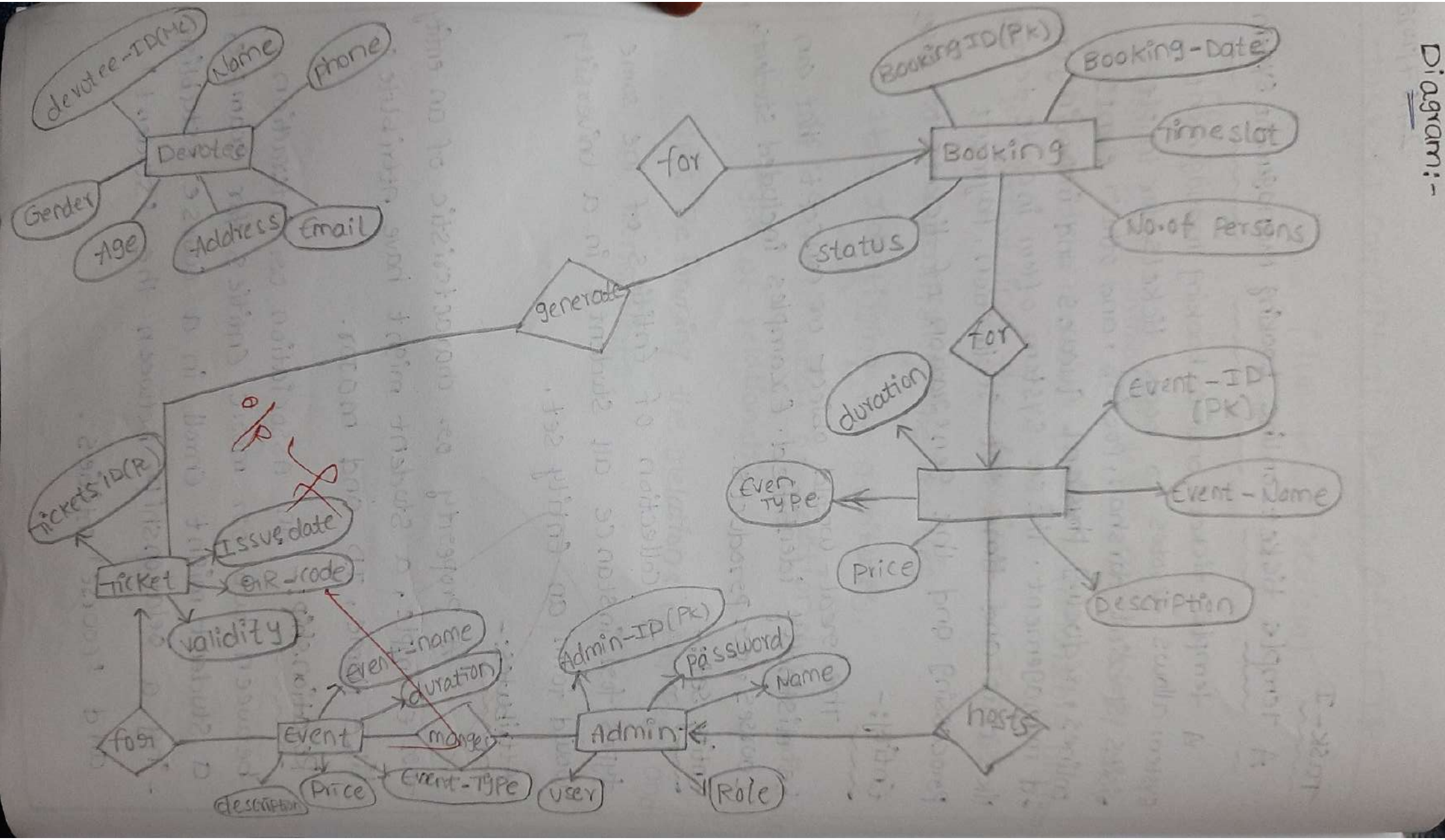
A collection of Entities of the same types for instance all students in a University would form an ~~Entity~~ set.

Attributes:-

A Property or characteristic of an entity for example, a student might have Attributes like Name, ID and major.

Relationship:- An Association or interaction between two or more Entities for example, a student might Enroll in a Course Establish-ing a Relationship between the 'student' and 'course' Entities.

Diagram:-


devotee-ID(PK)
Name
Phone
Devotee
Gender
Age
Address
Email
for
Booking
BookingID(PK)
Booking-Date
timeslot
status
No.of Persons
generate
for
duration
Event-ID (PK)
Event Type
Event-Name
Price
Description
ticketsID(P
Issue date
Ticket
QR-code
validity
Event-name
duration
Admin-ID(PK)
Password
Name
fosr
Event
manage
Admin
hosts
description
Price
Event-Type
User
Role

Aim:-

To design and develop an Entity-relationship (ER) model for a temple ticket online Booking management system that allows devotes to book tickets for temple visits, special darshans, poojas, and other events online, reducing physical queues and improving crowd management.

Attributes:-

=> Devotes:- Devote-ID, Name, Phone, Email, address Age, Grender.

=> Booking:- Booking-ID, Date, Time-slot, No.of-Persons, Status.

=> Events:- Event-ID, Event-Name, Description Price, Event-type, duration.

=> Temple:- Temple-ID, Temple-Name, location, opening-hours.

=> Payment:- Payment-ID, Amount, method, Status, Date.

=> ticket:- Ticket-ID, issue-Date, QR-code, validity.

=> admin:- Admin-ID, Name, username, Password, Role.

Types of Attributes:-

- simple : Name, Price
- composite : Address (street, city, state)
- multi-valued : phone
- Derived : Age (from DOB)

Relationships:-

- Devote → Booking (makes)
- Booking → Event (for)
- Booking → payment (has)
- Event → Temple (hosts)
- Admin → Event (manages)

Relationship Types:-

- one-to-one : Booking → payment, Booking → Ticket
- one-to-many : Devote → Booking
- Many-to-one : Booking → Event
- one-to-many : Temple → Event
- one-to-many : Admin → Event

Cardinality:- 1:N and 1:1 where applicable

Result:- The ER diagram for the temple ticket Booking system was successfully designed showing all entities, attributes, and Relationship with correct cardinalities for database implementations.

VEL TECH	
EX NO	1
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	3
RECORD (5)	5
TOTAL (20)	13+5=18
SIGN WITH DATE	

dh/7/25

31/7/25

Task 2: Generating design of other tradition -al database model.

Aim:- Creating Hierarchical/ Network model of the data base by enhancing the sound ab -stract data by enhancing the sound abstract data by performing following tasks using forms of inheritance.

2a. Identify the specificity of each relation -ship find and form surplus relations.

2b. check is a Hierarchy / has-a- hierarchy and performs generation and /or specialization relationship

2c. Find the domain of the attribute and perform check constraint to the applicable.

2d. Remove the relations.

2e. Perform Sql relations using DDL, DCL comma -nds.

2a. Identify the specificity of each relationshi -P, find and form surplus relations.

1) Relationship specificity:-

a. one-to-many (Devotees ⟷ Booking):

• Each devotee can have many bookings.

• Each booking is uniquely linked to one devotee

b. (Booking ⟷ ticket) one-to-one:

• one booking generates one or more tickets.

• Ticket is dependent on Booking

c. one-to-one (Booking ↔ Event

• Each booking is made for one Event

d. one-to-one or one-to-many

Payment <-> Booking:

• Each payment is tied to a booking

2. Surplus relations:-

Here we identify entities that might be redandant, merged or consolidated for normalization.

a. Ticket may be merged with booking (1:1 or 1:n)

b. Event details redundant in booking

c. Payment and Booking 2:2 Relation

2b. check is a hierarchy / has-a-hierarchy and Performs generalization and for specialization relationship:-

• devotee is a Person (if Extended to Priest, admen)

• Event / seva is a service of varous temple services are modeled

• Booking has-a ticket

• Booking has a relationship with user and temple.

Generalization:-

- Person (general) → devotee (specialized)
- Service → Event / seva

specialization:-

Different types of users: Devote user, Admin user.

2C. Attribute Domain & check constrains:-

- User phone-number: VARCHAR (15), CHECK (Phone-number LIKE [0-9]{10,15})
- Booking date-of-booking: DATE, CHECK (date-of-booking >= CURRENT-DATA)
- Ticket-seat-count: INT, CHECK (seat-count BETWEEN 2AND20)
- Event type: ENUM ('DARSHAN', 'PUJA', 'abhishe-kam'), CHEK (typeIN (..))

2d. Renaming Relations:

- User → Temple user
- Booking → Temple Booking
- Ticket → Temple Ticket
- Event → Temple Event

2e. SQL Relations using DDL · DCL

```
CREATE TABLE Temple User (
        User-id INT PRIMARY KEY,
        name VARCHAR (100) NOT NULL,
```

```
phone-number VARCHAR(15) NOT NULL
CHECK (Phone-number ~ '^[0-9]{10,15}$')
);

CREATE TABLE Temple (
    temple-id INT PRIMARY KEY
    temple-Name VARCHR(100) NOT NULL,
    location VARCHAR(255)
);

CREATE TABLE Temple Event (
    Event-id INT PRIMARY KEY
    temple-id INT REFERENCES
TEmple(Temple-id)
        Event-type VARCHAR(20) CHECK.
(event-type IN ('Darshan', 'PUJA', 'Abhishe
        -kam')),
    Event-date DATE
    CONSTRINT chk-Event-date CHECK
(Event-date >= CURRENT-DATE)
);

CREATE TABLE Temple Booking (
    booking-id INT PRIMARY KEY,
    user-id INT REFERENCES
Temple User(user-id)
```

```
Event-id INT REFERENCES
Temple Event(Event-id),
    booking-date DATE DEFAULT
CURRENT-DATE
);
CREATE TABLE Temple Ticket(
   -ticket-id INT PRIMARY KEY,
   booking-id INT REFERENCES
Temple Booking (booking-id),
 Seat-count INT CHECK (Seat-count
  BETWEEN 1 AND 10)
);
```

VEL TECH - CSE	
EX NO	9
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	3
RECORD (5)	+5
TOTAL (20)	
N WITH DATE	15+5=18

3/04/25

Result:-

Thus the Hierarchical model and Network model has been successfully Created.

7/8/2025

TASK 3:-

Using Clauses, operators and functions in queries.

Aim:- TO Perform query Processing on a temple ticket online Booking management system for different retrieval results of queries using DML, DDL operations with aggregate functions, data function, string function, set clauses and operators.

Temple:-

TEMPLE ID	NAME	LOCATION	CONTACT_NO
T1D01	Tirumala venkateshwar	Tirupati	9014276954
T1D02	meenakshi Amman	madurai	7989222725
T1D03	Kashi vishwanath	varanasi	7013982716
T1D04	Jagannath Temple	Puri	9701224404
T1D05	Golden Temple	Amritsar	9908616499

visitor:-

visitor ID	F.Name	t.Name	age	Email	contanct - No
V001	Anil	Kumar	30	Anil@gmail.com	9014276954
V002	Akash	Reddy	25	Akash@gmail.com	9701224404
V003	Priya	sharma	28	Priyas@gmail.com	9908290010
V004	Aarav	Patel	35	Aaravp@gmail.com	7013982716
V005	sneha	Rao	22	snehar@gmail.com	7989222725

Booking :-

Booking ID	TempleID	VisitorID	Booking-date	Ticket-Type	amount
B001	T1D01	V001	2024-06-15	VID	500
B002	T1D02	V002	2024-06-16	General	100
B003	T1D01	V003	2024-06-17	General	100
B004	T1D03	V004	2024-06-18	VIP	700
B005	T1D01	V005	2024-06-19	general	200

Priest:-

Priest ID	FName	LName	Age	Email	Contact.No.
P001	Ramesh	1year	50	ramesh@gmail.com	9014276954
P002	suresh	sharma	45	suresh@gmail.com	9701229404
P003	manish	Pas	40	manish@gmail.com	7989222785

2. Retrieve details of visitors whole first name starts with 'A'.

```
SELECT *
FROM visitor
WHERE FNAME LIKE 'A%';
```

visitor ID	FNAME	LName	Age	Contact-No
V002	Akash	Reddy	25	9701224404
V004	Aarav	patel	35	7013982716

3. Add a column for 'special-seva' in Booking table.

```
ALTER TABLE Booking ADD special seve
VARCHAR(50);
```

Result*

Table altered successfully

4. Count the number of VIP ticket bookings.

```
SELECT COUNT(*)
FROM Booking
WHERE Ticket_TYPE = 'VIP';
```

Resuct:-

```
count(*)
  2
```

5. Display temple detail for temple IDS: 'TIDO1', 'TIDO3' and 'TIDO4'.

```
SELECT *
FROM TEMPLE
WHERE TEMPLE ID IN ('TIDO1', 'TIDO3', 'TIO4');
```

Result:-

temple ID	Name	location	contact_NO
TIDO1	Tirumala	Tirupati	9701224404
TIDO2	kashi	varanasi	7013952716
TIDO3	Golden temple	Amritsar	7989222725.

6. select visitor ID and names of visitor who book and 'special' tickets.

```
Select visitor ID, FNAME, LName
FROM visitor
WHERE visitor IDIN (
SELECT visitor ID FROM Booking WHERE Ticket_
TYPE = 'special'.
);
```

Result:-

Visitor ID	FName	LName
V005	Sneha	Rao

7. find the Priest ID of Priest who have not been assigned any temple.

```
SELECT Priest ID
FROM Priest
WHERE Temple ID is Null;
```

Result;

Priest ID

(No Result if all Priests are assigned).

VEL TECH - CSE	
EX NO	8
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	3
RECORD (5)	5 = 18
TOTAL (20)	
SIGN WITH DATE	

Result:- Thus, query Processing for temple ticket online Booking Management system using clauses operators, and functions was successfully Performed.

TASK-4:-

Using functions in Queries and writing sub Queries:-

Aim:- To Perform advanced query processing and test its heuristics by designing optimal correlated and nested subquires such as finding summary statics in the temple ticket online Booking Management System.

4.1: To retrieve all temple details including the count of bookings for each temple.

```
SELECT t.Temple ID,
       t.Name AS Temple Name,
       t. location,
       t. Contact-No,
COUNT (b.Booking ID) AS Total Bookings.
FROM Temple t
LEFT JOIN Booking b
ON. t.Temple ID = b. Temple ID.
GROUP BY t. Temple ID, tName, t. location, t. Conta
-Ct-No;
```

output:-

Temple ID	Temple Name	location	Total Bookings
T1D01	Tirumala	Tirupati	3
TID02	meenakshi Annam	madurai	1
TID03	Kashi	varanasi	1
TID04	Jagannath temple	Puri	0
TID05	Golden temple	Amritsar	0

4.2 : To retreive the total number of 'special' seva booking in a temple - wise manner.

```
SELECT t.Name AS Temple Name,
 COUNT(*) AS Total special Bookings
 FROM Templet
 JOIN Booking b
 ON t.TEMPLEID= b.Temple ID
 WHERE b.Ticket_Type = 'special'
 Group By t.Name;
```

output:-

Temple Name	Total special Bookings
Tirumala venkateshwara	1

4.3 : To Retrieve the details of temples where bookings include 'VIP' tickets.

```
SELECT*
FROM Temple
WHERE Temple ID INC
  SELECT TempleID
  FROM Booking
WHERE Ticket_Type = ('VIP')
);
```

output:-

TempleID	Name	location	contact-No
T1D01	Tirumala	Tirupati	9701224404
T1D03	Kashi	varanasi	7013982716

4.4: To retrieve visitors and booking details of visitors who are above 25 year old.

```
SELECT v.visitor ID,
       V.FName AS visitor Name,
       v.age,
       b.Booking ID,
       b.Booking_Date,
       b.Ticket_TYPE,
       b.amount.
FROM visitor v
JOIN Booking b
ON v.visitor ID = b.visitor ID.
WHERE v:Age = 25;
```

Output:-

visitor ID	visitor Name	Age	Booking ID	Amount
V001	Anil	30	B001	500
V003	Priya	28	B003	100
V004	Aarav	35	B004	700

4.5: To retrieve the details of temples with no bookings.

```
SELECT *
FROM TEMPLE
WHERE Temple ID NOT IN (
 SELECT Temple ID
 FROM Booking
 );
```

output:-

TempleID	Name	location	contact-No
TID04	Jagannath temple	Puri	9701224404
TID05	Golden temple	Amritsar	9014276954

4.6: To retrieve the templeid, Temple name, and visitor name for a Particular visitorid given

```
SELECT t.TempleID,
       t.Name AS TempleName,
       v.FName AS visitorName
FROM Temple t
JOIN Booking b
ON t.TempleID = b.TempleID
 JOIN visitor v
ON b.visitorID = v.visitorID
WHERE v.visitorID = 'V005';
```

output:-

TempleID	TempleName	visitorName
TID01	Tirumala	Sneha

Result:- Thus, the queries using function, joins, and nested subquries were successfully executed for the temple ticket online Booking management system.

14/8/25

VEL TECH - CSE	
EX NO	4
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	4
RECORD (5)	+5
TAL (20)	19
N WITH DATE	

TASK NO:-05 20/8/25

writing Join Queries ,equivalent & for recursive

Aim:- TO Perform advanced query Processing and test heuristics using optimal correlated and nested subqueries such as retrieving summary statics and ticket booking details for the online temple ticket Booking management system.

5.1) To Retrieve all temples and their available tickets.

```
SELECT tm.Name as temple , tk.Ticket ID , tk.TYPE,
  TK.Price
FROM Temple tm
JOIN Ticket tk ON tm.Temple ID = TK.Temple ID;
```

5.2) To list all Bookings along with temple name and devotee name.

```
SELECT b.Booking ID , d.Devotee Name , tm.Name As
   Temple Name , tk.TYPE , b.Booking Date ,
FROM Booking b
 JOIN Devotee d ON b.Devotee ID = d.Devotee ID.
 JOIN Ticket tk ON b.Ticket ID = tk.Ticket ID
 JOIN Temple tm ON TK.Temple ID = tk.Temple ID;
```

5.3) To count the number of bookings made for each temple.

```
SELECT tm.Name AS Temple Name,
 COUNT (b.Booking s) AS Total Bookings
 FROM Temple tm
LEFT JOIN Ticket tk ON Tm.temple ID = tk.Temple ID
```

Output :-

Temple	PoojaName	Price.
Tirupathi temple	SuPrabhata seva	300
Tirupathi temple	Archana	100
kanchi temple	Abhishekam	500
maduri temple	special Darshan	250

Output :-

Booking ID	Devotee	PoojaName	Booking date	slot time	Status
B101	Rajesh	SUPrabhat seva	22-Jun-23	6:00Am	confirmed
B102	Janu	Archana	22-Jun-23	7:30Am	confirmed
B103	Arjun	Abhishekam	23-Jun-23	8:30Am	confirmed
B104	Kavitha	Special darshan	23-Jun-23	8:30Am	confirmed

Output

Devotee	total booking
Rajesh.	1
Janu	1
Arjun	1
Kavitha	1

Output

Devotee ID	FName	mobile no
D103	Arjun	93847679
D102	Janu	93982472

```
LEFT JOIN Booking b ON tk.Ticket ID = b.Ticket ID.
Group BY tm.Name;
```

5.4) To find All devotees who booked tickets for Tirupati Balaji?

```
SELECT d.Devotee ID, d.Devotee Name, d.Contact No,
       tm.Name AS Temple
FROM Devotee d
JOIN Booking b ON d.Devotee ID = b.DevoteeID.
JOIN Ticket tk ON b.Ticket ID = tk.Ticke t ID
JOIN Temple tm ON tk.temple ID = tm.Temple ID.
WHERE tm.Name = 'Tirupati Balaji';
```

5.5) To retrieve all devotees details including total tickets booked.

```
SELECT d.Devotee ID, d.Devotee Name, d.contact
  NO, COUNT (b.Booking ID) AS Total Tickets
FROM Devotee d
LEFT JOIN Booking b ON d.Devotee ID = b.Devotee ID
Group BY d.Devotee ID, d.devatee Name, d.ContactNo
```

5.6) To retrieve the total number of special Darshan tickets booked temple-wise

```
SELECT tm.Name AS Temple,
    Count (b.Booking ID) AS Special Darshan
    Count.
    FROM TEMPLE tm
JOIN Ticket tk ON tm.temple ID = TK.Temple ID
```

output

Pooja name	Total Booking
Suprabhata seva	1
Archana	1
Special darshan	1
Abhishekam	1

output

Temple	cancelled Bookings
Kachi temple	1

5.8

out put

FName	Age	BookingID	Booking Date	slot time
Janu	22	B102	24-Jun-23	6:30Am

Status
confirmed

5.9

output

Temple ID	Name	location	Head priest
T004	Rameshaswaram temple	Rameshwaram	Vishnu Devan

5.10

out put

Temple	Poojaname	Devotee	Booking Date	slot time
Tirupathi temple	Archang	meena	22-Jun-23	9:00Am

```
JOIN Booking b ON TK Ticket ID = b.Ticket ID
WHERE tk.TYPE = 'Special Darshan'
GROUP BY tm.Name;
```

5.7) To retrieve the temple details where ticket are still available (not fully booked)

```
SELECT tm.Temple ID, tm.Name, tK.TYPE, tk.Price
FROM Temple tm
JOIN Ticket tk ON Tm.temple ID = tk.Temple ID
WHERE tK.Available seats > 0;
```

5.8) To Retrieve devotee and Booking details for devotee above 40 years.

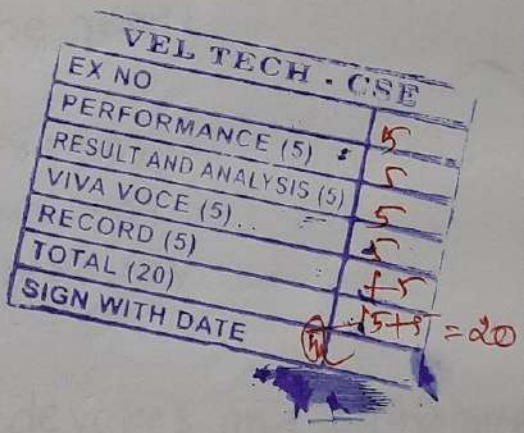
```
SELECT d.Devotee ID, d.Devotee Name, d.Age,
b.Booking ID, b.Booking date, tm.Name AS Temple
FROM Devotee d
JOIN Booking b ON d.Devotee ID = b.Devotee ID
JOIN Ticket tk ON b.Ticket ID = tk.Ticket ID
JOIN Temple tm ON tk.temple ID = tm.Temple ID
WHERE d.Age > 40;
```

5.9) To retrieve temple details where no book-ings have been made.

```
SELECT * FROM temple
    WHERE temple ID NOT IN (SELECT TK.Temple
                            I D)
                FROM Ticket tk
JOIN Booking b ON Tk.ticket ID = b.ticket ID);
```

5.10) To Retrieve temple ID, Ticket ID, temple name and devotee name for a particular booking ID given

```
SELECT tm.Temple ID, tk.Ticket ID, tm.Name As
        Temple Name, d.Devotee Name
FROM Temple tm
JOIN Ticket tk oN tm.Temple = tk.Temple ID
JOIN Booking b oN tk.Ticket ID = b.Ticket ID
JOIN Devotee d oN b.Devotee ID = d.Devotee ID
WHERE b.Booking ID = 'B002';
```



20/8/25

Result:- Thus, the Queries using Join Quries, Nested Queries, and Equivalent Queries were Executed successfully for the online Temple Ticket BMS.

Task6 :- Procedures, function and loops temple online Booking.

Aim:-

1. write PL/SQL block that calculates the average age of devotees and displays the result.
2. write a PL/SQL black that inserts a new booking into the booking Table
3. create a function that returns the total number of bookings for a Particular Temple
4. write a non-recursive PL/SQL Procedure to retrieve even-numbered booking IDS registered for any seva.

(1) PL/SQL black that calculates the average age of devotees and displays the result.

```
DECLARE
  v_total_age  Number := 0;
  v_count_devs Number := 0;
  v_avg_age    Number := 0;
BEGIN
  -- loop through all devotees and aggregate age.
  for rec IN (SELECT AGE FROM Devotee) loop
    v_total_age := v_total_age + DVC (rec.AGE8);
    v_count_devs := v_count_devs + 1;
  END loop;
  If v_count_devs > 0 THEN.
    v_avg_age := v_total_age / v_count_devs;
  END IF;
```

```
-- Displaying the result
DBMS_OUTPUT.PUT_LINE ('total Devotees: ' ||
v_count_devs);
DBMS_OUTPUT.PUT_LINE ('Total Age : ' ||
v_total_age);
DBMS_OUTPUT.PUT_LINE ('Average Age : ' ||
TO_CHAR (ROUND (v_avg_age, 2)));
END;
/
```

2) PL/SQL block that inserts a new booking into the Booking table

```
DECLARE
    v_BookingID VARCHAR2(10) := '&BookingID';
-- e.g; 'BKG203'
    v_DevoteeID VARCHAR2(10) := '&DevoteeID';
-- e.g., 'DEVO45'
    v_TempleID VARCHA2(10) := '&TempleID';
-- e.g, 'TMPOOL'
v_BookingDate DATE :=
TO_DATE ('&B_DATE', 'YYYY-MM-DD'); --
e.g., 2025-09-01
v_slot VARCHAR2(20) := '&slot'; --
e.g, '08:00-09:00'
v_sevatype VARCHAR2(30) := '&sevatype';
-- E.g; 'Archana'
v_Amount Number(10,2) := &Amount;
-- e.g; 250.00
```

```
v-status VARCHAR2(15):= '&status';
BEGIN
   INSERT INTO Booking
   (Booking ID, Devotee ID, Temple ID, Booking Date,
   v-Slot, v-Sevatype, v-Amount, v-status);
    COMMIT;
DBMS-OUTPUT.PUT-LINE ('Booking record
inserted successfully:');
   EXCEPTION
     WHEN OTHERS THEN
     DBMS-OUTPUT-PUT-LINE ('Error:' || SQLERRM);
    ROLL BACK;
     END;
    /
```

3) Function to return total number of bookings in a particular Temple

```
CREATE OR REPLACE FUNCTION
Get total BookingsIN Temple (P-Temple ID
VARCHAR2)
RETURN NUMBER
IS
 v-Total Number;=0;
 BEGIN
   SELECT count(*) INTO v-total
     FROM Booking
     WHERE Temple ID=P-temple ID;
     RETURN v-TOTAL;
     EXCEPTION
        WHEN NO-DATA-FOUND THEN
          RETURN 0;
```

```
WHEN OTHERS THEN
    RETURN -1;
END Get total Bookings IN Temple;
/
Test the function
DECLARE
    v-res NUMBER;
BEGIN
    v-res := Get total Bookings in temple
    ('TMPOOT');
DBMS-OUTPUT.PUT-LINE ('No of bookings in
temple :' || v-res);
    END;
/
```

4) Non - recursive Procedure to retrieve even-numbered Booking IDS

```
CREATE OR REPLACE PROCEDURE
Get Even Numbered Booking ID'S
IS
BEGIN
    FOR rec IN
    (
    SELECT Booking ID
    FROM Booking
WHERE MOD (TO-Number (Booking ID), 2) = 0
    )
    LOOP
DBMS-OUTPUT.PUT.LINE ('Even-Numbered
Booking ID:' || rec.Booking ID);
    END LOOP;
```

1. sample output

Total Devotees : 120
Total Age : 3060
Average Age : 25.5

2. Sample output

Enter value for Booking ID : BKG203
Enter value for Devotee ID : DEV045
Enter value for Temple ID : TMP001
Enter value for BDate : 2025-09-01
Enter value for Slot : 06:00-07:00
Enter value for sevaType : Archana
Enter value for Amount : 250.
Enter value for status : CONFIRMED
Booking record inserted successfully.

3. output

No. of bookings in temple : 42

4. output

Even-Numbered BookingID : 102
Even-Numbered BookingID : 204
Even-Numbered BookingID : 306.

o/p

```
END Get Even Numbered Booking IDS;
/
Run the Procedure
    BEGIN
      Get Even Numbered Booking IDS;
    END;
/
```

EX NO	6
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	5
TOTAL (20)	5+5=20
SIGN WITH DATE	

Result:- Thus the PL/SQL Procedures, functions and Loops on the temple online Booking management system were successfully implemented and tested

7.1 output

row inserted into ticket Both

Triggers fired → New payment record insert into payment table

7.2

Exception ex;

select * from below Booking data.

output

Devotee ID Devotee Name Email

temple name ticket ID Booking Date visit Date

No.of tickets Amount

D501 Rahul sharma/rahul@gmail.com

Tirupathi balaji T101 01-sep-25 10-sep-25

2 200

7.3

EXCEPTION ex;

BEGIN:

GET Event ticket for temple ('TEMPD1');

END;

/

O/P

Even Ticket ID : T101

Even ticket ID : T104

TASK 7: Triggers, views and exceptions

Aim:- To conduct events, views, exceptions on CRUD operations for restricting phenomenon in a temple online Booking management system.

a) To create a trigger in PL/SQL that automatic -ally inserts a new record in the booking-audit table when a new record is inserted into the bookings table.

b) To create a view that displays the details of bookings along with the user and temple details.

c) To write a non-recursive PL/SQL Procedure to retrieve even-numbered Booking IDs registered for any temple.

a) trigger

To create a trigger in PL/SQL that automati -cally inserts a new record in the booking-audit table when a new record is inserted into the bookings table.

```
CREATE OR REPLACE TRIGGER
    insert-booking-audit
    AFTER INSERT ON bookings
    FOR EACH ROW
    BEGIN
INSERT INTO booking-audit (audit-id,
booking-id, user-id, temple-id, slot-id,
new-status, audit-ts)
    VALUES
```

```
(booking_audit_seq.NEXTVAL, :new.booking_id,
 :new.user_id; :new.temple_id; :new.slot_id,
Nvt (:new.status, 'PENDING'), SYSDATE);
 END;
 /
```

b) view

To create a view that displays the details of bookings along with the user and temple details.

```
CREATE OR REPLACE VIEW BOOKING DETAILS AS
SELECT b.booking_id,
       u.user_id,
       u.full_name AS user name,
       u.mobile AS users mobile,
       t.temple_id,
       t.temple_name,
       s.slot_id,
       s.slot_time,
       b.status,
       b.created_at
FROM bookings b
 JOIN users u ON b.user_id = u.user_id JOIN
temples t ON b.temple_id = t.temple_id
 JOIN slots s ON b.slot_id = s.slot_id;
```

To display:

```
SELECT * FROM BOOKING DETAILS;
```

c) Procedure

To write a non- recursive PL/SQL Procedure to retrieve even -numbered Booking IDS registered for any temple.

```
CREATE OR REPLACE PROCEDURE
   Get Even Booking IDS For Temple(
   in- temple- id   IN Number,
   out- Even -booking-ids OUT
     SYS.ODC IN NUMBER UST
     )AS
  BEGIN
   OUt - Even -booking-ids :=
     SYS.ODC IN NUMBER LIST();
FOR rec IN (SELECT booking-id
       FROM bookings.
    WHERE temple- id = ID- temple -id
    AND MOD (booking- id,2) =0)
loop
   out- Even -booking -ids.EXTEND;
OUT - Even -booking-ids(out- Even -booking
 -ids.COUNT) := rec.booking-id;
      END loop;
      END;
      /
```

Exception Example:-

```
DECLARE
    temple_id NUMBER := 10;
    even_ids SYS.ODCINUMBERLIST;
BEGIN
    Get Even Booking IDS For Temple (temple_id,
even_ids);
  FOR i IN 1... even_ids COUNT LOOP
  DBMS_OUTPUT.PUT_LINE ('Even Booking
ID:' || even_ids (i));
      END LOOP;
      END;
  /
```

VEL T.	
EX NO	7
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	5
TOTAL (20)	15+5=20
SIGN WITH DATE	

4/9/25

Result:- Thus the Triggers, views and Exceptions experiment for Temple online Booking management system was successfully completed and result are verified.

Task-8 :- CRUD operations in Document databases.

Aim :- To perform CRUD using NPm design on mango DB for Temple online booking management system, deigning a document database and performing CRUD operations like creating, inserting, querying, finding, updating and deleting.

Steps :-

step-1 :- Install mango DE wing the following link:
https://www.mongodb.com/try/download/community

step-2 :- Install mangosh using the below link:
https://www.mongodb.com/docs/mongodb shell/#download-and-install-mangosh.

step-3 :- To add mango DE shell binary's location to your path environment variable.

- 3.1 open the control panel
- 3.2 go to System and security → system
- 3.3 Click Advanced System settings
- 3.4 Click environment variables
- 3.5 In system variables, select path and click edit.
- 3.6 Add the filepath to your mongo sh. binary.
- 3.7 Click ok to confirm.

step-4 :- open a command prompt and type mangosh --- help to confirm installation.

step-5 :- open mango shell 4.0 from c:\program files\mango DB\server\bin\mangod.exe

step-6 :- Type the CRUD (Create, Read, update, Delete) Commands in mango DB.

CRUD operations

1. Create collection

db.Createcollection ("Temple Booking")

2. output :

{"acknowledged": True, " inserted id":
objected("651c1f726befe7993adf901")}

3. output:

{
"acknowledged": True,
"insertedIds":[
objectid ("651c1f726bebfe7993adf902"),
objectid("651c1f726bfe7a93adf903"),
objectid("651c1f726bebfc7993adf900")
]
}

4. output

{"_id": objectid("651c1f726bcbfe7a93adf901"),
"Booking ID": "B001", "Temple Name": "Sri Venkateshwara
Temple", "Location": "Tirupathi", "Devotee Name":
"Ramesh", "Phone": 9876543210, "Booking Date": "2025-09-10"}
{"_id": objectid("651c1f726bebfe7993adf902")
"Booking ID": "B002", "Temple Name": "Meenakshi Amman
Temple", "Location": "Madurai", "Devotee Name": "Suresh",
"Phone": 9876501234, "Booking Date": "2025-09-12"}

{"ok":1}

2. Insert one record

db.TempleBooking.insertone(

{BookingID:"B001", Templename:"Sri venkateswara Temple", Location:"Tirupati", Devotee Name:"Ramesh", Phone:9876543210, Booking Date:"2025-09-10"},

)

3. Insert Many records

db.TempleBooking.insertmany([

{BookingID:"B002", Temple Name:"Meenakshi Amma Temple", Location:"Madurai", Devotee Name:"suresh", phone:9876501234, Booking Date:"2025-09-12"},

{BookingID:"B003", Temple Name:"kapaleeshwarar Temple", Location:"chennai", Devotee Name:"priya", phone:9966332211, Booking date:"2025-09-15"},

{BookingID:"B004", Temple Name:"Annanalaiyar Temple", Location:"Thiruvannamalai", Devotee Name:"Anitha", phone:9955886644, Booking date:"2025-09-20"}

])

4. find records

db.TempleBooking.find()

5. Pretty format find

db.TempleBooking.find().pretty()

6. update a record.

db.TempleBooking.updateone({

5. Out Put :-

{ "_id": Objectid ("6516ff72bebfe7993adf 903"),

"Booking ID": "B003",

"Temple Name": "kopaleeshwarar Temple",

"Location": "chennai",

"Devotee Name": "priya",

"phone": 9966332211,

"Booking Date": "2025-09-15"

}

6. Output :-

{ "acknowledged": True, "matched Count": 1,

modified Count": 1}

7. Out Put :-

{ "acknowledged": True, "delete Count": 1}

{ Booking ID: "B003" },
{ $set: { Phone: 9123456789,
Devotee Name: "Priya Sharma" } }
)

7. Delete a record

db.TempleBooking.deleteOne({ Booking ID: "B001" })

VEL TECH - CSE	
EX NO	8
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	4
RECORD (5)	+5
TOTAL (20)	14+5=19
SIGN WITH DATE	T.M.Thr 11.09.24

Result :- Thus, CRUD operations using NPM design of MongoDB for Temple online Booking Management System were successfully implemented, including creating, inserting, querying, updating, and deleting records.

Output :-

Devotee	Temple	Seva	Date
Ravi	Sri venkateswara temple	Archana	2025-09-20

output :-

BOOKING ID	Updated Seva.
5001	Abhishekam.

TASK:-9:

CRUD operations in graph database.

Aim:- TO Perform graph QL/Neo; graph space design for commendation engies. Also perform CRUD operation like creating inserting, Querying, finding deleting operations on graph spaces.

1. Create operation.

TO Create nodes and relationships for Devotees, temples, and bookings.

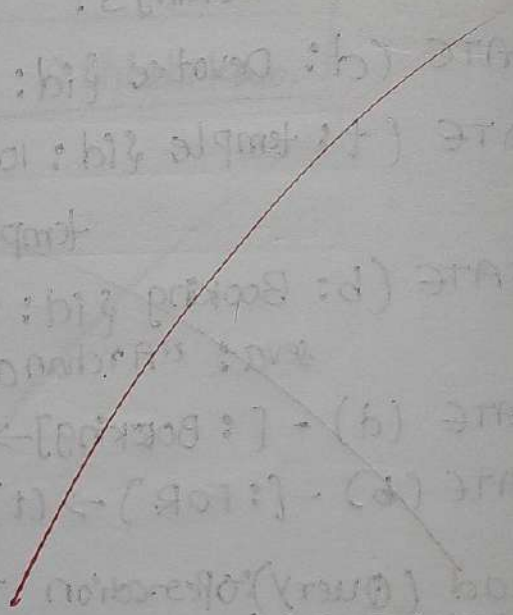
```
CREATE (d: Devotee {id: 1, name: "Ravi", age: 30})
CREATE (t: temple {id: 101, name: "Srivenkateswar temple", location: "Tirupathi"})
CREATE (b: Booking {id: 500); date: "2025-09-20", seva: "Archana"})
CREATE (d) - [: BOOKing] -> (b).
CREATE (b) - [: FOR] -> (t);
```

2. Read (Query) operation TO find which temple devotee has booked.

```
MATCH (d: devotee
{name: "Ravi"}) - [: BOOKED] -> [b: Booking] -
[: FOR] -> (t: temple]
RETURN d: name AS Devotee, t. name AS Temple.
b. seva AS seva, b. data AS Date.
```

delete output:-


```
{
"date" : {
  "delete Booking" : {
    "success." : true,
    "message" : "Booking deleted successfully"
    }
  }
}
```

3. Update operation :-

To update booking details (example : seva . type).

```
MATCH (b:Booking {id: 5001}).
SET b.seva = "Abhishekam"
RETURN b.id AS .Booking ID, b.seva AS .UPDATE seva;
```

Delete operation:-

To Delete a booking when the devotee cancels it

```
MATCH (d: Devotee
{name: "Ravi"}) -[r: BOOKED] -> [b:Booking {id: 5001}]
DETACH DELETE b
RETURN "Booking Delete "AS . status;
```

Graph :-

```
mutation {
  delete Booking (id: "5061") {
  success
    message
    }
}
```

VEL TECH - CSE	
EX NO	9
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5	5
VIVA VOCE (5)	4
RECORD (5)	+5
TOTAL (20)	18
SIGN WITH DATE	18.09.24

Result:- CRUD operations were successfully implemented for the online temple booking management system. The system allowed creating reading., updating and deleting booking using graph data base Queries effectively

Output :-

User ID	User Name	User phone	Temple ID	Temple Name	Temple location
101	Ramesh	9876543210	1	Tirupathi	Andhra Pradesh

Event ID	event name	Event Date	SeatNo	ticket ID	Price
501	Special Darshan	2025-10-01	A1	9001	300

Booking date
2025-09-20

Result (2NF)

User ID	ticket ID	Booking date
101	9001	2025-09-20

TASK - 10:-

Normalizing database using functional dependencies up to third Normal form.

Aim:- TO normalize the below relation and create the simplified tables with suitable constrains for the online temple booking management system.

Given Relation:-

Temple Booking (
Booking Id, User Id, UName, U email, U contact, TempleID, T Name, T location, Priest - ID, priest Name, Priest Contact. Ticket ID, Booking Date, Darshan Date.
)

step(a): APPLY functional dependencies -> Normalize to 1NF.

first, identify functional dependencies (FDs):

1. User ID -> UName, U email, U Contact
2. Temple ID -> T Name, T location
3. Priest ID -> Priest Name, Priest contact
4. Ticket ID -> Ticket type, Price
5. Payment ID -> Payment mode, Payment status, Transaction Date.

Step(b):- Normalize using FD+ & α + candidate keys:

- Booking ID (main unique identifier of a Booking).

FROM α + (closure of candidate keys):-

- Booking ID* = {All attributes} -> Confirm Booking ID is a candidate key.

Event-T :-

Event ID	Event Name	Event date	Temple ID
E1	Aadi festival	2025-10-01	T1

Ticket T:-

Ticket ID	Event-ID	Seat NO	Price
TK1	E1	A1	100.00

Booking -2NF:-

User ID	Event ID	Seat NO	Booking Date	Ticket ID
U1	E1	A1	2025-09-01	TK1

Booking - BCNF :-

Booking ID	User ID	Ticket-ID	Booking date
1	U1	TK1	2025-09-01

• User ID, Temple ID, Priest ID, Ticket ID, Payment ID also act as foregin keys with their own depend -encies.

step (c):- minimal cover / canonical cover :-

1. User ID → UName, U email
2. Temple ID → T Name, T location
3. Priest ID → Priest Name, Priest contact
4. Ticket ID → Ticket type, Price.

step(d):- Normalize to 2NF :-

Conditions for 2NF

→ Must already be in 1NF

→ No Partial dependency

the schema is in 2NF

step (e):- Normalize to 3NF :-

Condition for 3NF

→ Must already be in 2NF

→ No transitive dependencies.

check:

* Booking ID → User ID → User Name
* Booking ID → Temple ID → T Name
* Booking ID → Priest ID → Priest Name.

Final simplified tables in 3NF :-

① User table :-

User (user ID [PK], U Name, U Email, u contact).

② Temple table :-

~~Priest [Priest ID [PK], Priest Name, Priest Contact).~~

Temple (temple ID [PK], T Name, T location)

③ Priest table :-

priest (Priest ID [PK], Priest Name, Priest (contact))

④ Ticket table :-

Ticket (ticket ID [PK], Ticket type, Price)

Constraints :-

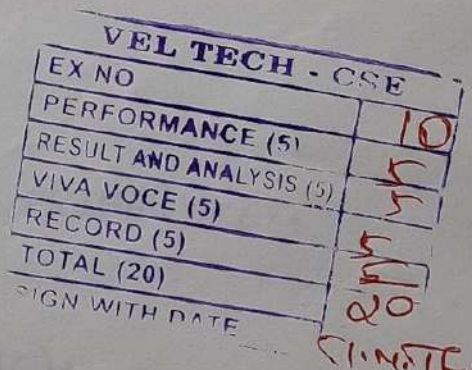
-> Primary Keys: Booking ID, User ID, Temple ID, Priest ID, Ticket ID, Payment ID.

-> Foregin Keys :-

(i) Booking . User ID -> User User ID

(ii) Booking . Temple ID -> Temple . Temple ID

(iii) Booking . Priest ID -> Priest Preist ID



Result:- Thus, the given relation for the online temple ticket Booking management system has been normalized into simplified tables up to third Normal form (3NF) & improving data integrity.

TASK-11.

Menus, forms and Reports

Aim:- To design and implement an online temple Booking management system by creating and debugging menus, forms, and reports using Oracle forms, SQL server management studio, SQLite, or Net Beans, in order to manage devotee details, Pooja bookings, and generate booking reports efficiently.

Objective:-

To create and debug menus, forms, and reports for an temple ticket booking.

Description:- this system manages devotee registrations, Pooja bookings, and generates reports for temple authorities.

Data base design:-

1) Devotee table:-

```
CREATE TABLE Devotee(
    Devotee_ID INTEGER PRIMARY KEY,
    Name VARCHAR(50),
    mobile_No VARCHAR(15),
    Email VARCHAR(50)
);
```

2) Booking Table:-

```
CREATE TABLE Booking (
 Booking_ID INTEGER PRIMARY KEY,
 Devotee_ID INTEGER,
 Pooja_Name VARCHAR(50),
 Booking_Date DATE,
 Amount DECIMAL(10,2),
 FOREIGN KEY (Devotee_ID) REFERENCES
Devotee(Devotee_ID)
);
```

Menu Design:-

menu	options
Devotee	Add, Update, Delete Devotee.
Booking	New Booking, cancel Booking.
Reports	Booking Report, callection Report
Exit	close Application.

Form Design:-

Devotee Form:

Fields - Devotee_ID, Name, Mobile_No, Email.

Buttons - save, Update, Delete, clear.

Booking Form:

Fields - Booking_ID, Devotee_ID, Pooja_Name, Booking_Date, Amount

output:-

Devotee Table:-

Devotee -ID	Name	mobile-No	Email
101	Ramkumar	9876543210	Ram@gmail.com
102	Sita Devi	9876501234	sita@gmail.com

Booking Table:-

Booking - ID	Devotee - ID	Pooja - Name
201	101	Ganapathi Homam
202	102	lakshmi Pooja.

Buttons – Book, cancel, viewall.

Report Query:-

```
SELECT D.Name, B.Pooja_Name, B.Booking_Date,
B.Amount
FROM Booking B
JOIN Devotee D ON B.Devotee_ID = D.Devotee_ID
ORDER BY B.Booking_Date;
```

VEL TECH - CSE	
EX NO	11
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	5
TOTAL (20)	20
SIGN WITH DATE	09.10.24

Result:- the online temple Booking management System was successfully created with menu, forms and reports..

Mini Project: USE CASE-3:

A gift coupon application handles offers payment related information

BATCH MEMBERS

VTU30148	SUNKESWARA SUSHANTH
VTU29776	SHAIK GOUSE MOHIDDIN
VTU30639	DODLA SANTHOSH
VTU30464	MANNURU MEDHINI
VTU29154	KALLURI PEDDA BABU
VTU28548	CHANDRAJIT KAMALAKKANNAN
VTU27557	VADAMADHURA DILEEP KUMAR
VTU30543	YARASANI TARUN KUMAR REDDY
VTU30547	KOMMINA SRUTHI
VTU30538	GOSU BHARGAV RAM
VTU27624	SHAIK SHAHIL ROHAN
VTU28996	THONDAPU SAI JASWANTH
VTU28657	DASARI KOTISURYA
VTU27974	RAVURI NAGA DURGA PRASAD
VTU30565	BOARD OF SECONDARY EDUCATION ANDHRA PRADESH
VTU29377	DADIPINENI BHARGAVI
VTU27736	PANAGANTI SAI SRINATH
VTU28933	CHAMARTHI TEJESWAR RAJU
VTU30539	NAGANABOYINA MOHAN VENKAT
VTU29344	KANUPARTHI POOJITH REDDY
VTU27547	KONDAPATURI MADHU SUMANTH
VTU29764	KUCHANA SRAVIKA
VTU29362	VEPURALA VIPIN RAJ
VTU27789	GANAPATHIRAJU JAGANNADHA VARMA
VTU28867	KODURU KESAVA
VTU29367	GANDLURU VENKATA ARAVIND
VTU27479	ANISETTI ROHITHA

- **Aim:**
  To design and implement a Gift Coupon Application using a Relational Database System capable of horizontal scalability while maintaining ACID properties. The system must handle offer and payment-related transactions without data inconsistency or deadlocks and ensure that all relational tables are normalized up to 3NF or BCNF for efficient and reliable coupon management.
- **Algorithm:**

1. **Start Transaction**

2. **Validate Coupon Code** — Check if the coupon exists, is active, and within its validity period.
3. **Check Availability** — Verify total quantity and per-user usage limits.
4. **Lock Rows** — Use SELECT ... FOR UPDATE to prevent concurrent access.
5. **Insert Payment Record** — Create a pending payment entry.
6. **Insert Redemption Record** — Log the coupon usage for the user and merchant.
7. **Update Counters** —
   - Decrement total coupon quantity.
   - Increment user's coupon usage count.
8. **Mark Payment Complete** after success.
9. **Commit Transaction** if all checks pass; otherwise **ROLLBACK**.
10. **End Transaction**.

Properties

- Maintains **ACID** (Atomicity, Consistency, Isolation, Durability).
- Prevents **double spending** and **deadlocks** using consistent lock order.
- Normalized schema ensures no redundancy (up to **BCNF/4NF**).

Sample program:

```
-- Step 1: Table Creation (Normalized up to 3NF/BCNF)

CREATE TABLE users (
    user_id BIGINT PRIMARY KEY,
    username TEXT NOT NULL UNIQUE,
    email TEXT NOT NULL UNIQUE
);

CREATE TABLE merchants (
    merchant_id BIGINT PRIMARY KEY,
    name TEXT NOT NULL
);

CREATE TABLE coupons (
    coupon_id BIGINT PRIMARY KEY,
    code TEXT NOT NULL UNIQUE,
    merchant_id BIGINT REFERENCES merchants(merchant_id),
```

```
    PRIMARY KEY (coupon_id, user_id)
);
```

- **Output:**

Output (Sample Result)

username	coupon_code	discount_amount	final_payment	payment_status	remaining_stock
Alice	WELCOME50	50.00	200.00	COMPLETED	99

Result

- The Gift Coupon Application successfully performed the **coupon redemption transaction** maintaining **ACID** properties.
- The schema was **normalized up to BCNF/4NF,** eliminating redundancy and maintaining referential integrity.

The system ensured:

- **No data inconsistency**
- **No double redemption**
- **Sale concurrent access**
- **Horizontal scalability** through distributed SQL (e.g., CockroachDB or YugabyteDB)
- Thus, the relational database transacts efficiently without deadlocks and sustains up to **BCNF/4NF normalization,** perfectly suitable for handling **payment and offer-related data** in a gift coupon system.

VEL TECH - CSE

NO.	44
FORMANCE (5)	5
L AND ANALYSIS (3)	5
A VOCE (3)	3
ORD (4)	5
(15)	18
DATE	16/10/25

16/10/25